

```
// Settings live ONLY in this phone's  
browser storage (localStorage).  
// They are never part of the GitHub  
repository, so no key can be  
committed by accident.
```

```
export const PROVIDER_PRESETS = {  
  groq: {  
    label: "Groq (free plan)",  
    type: "openai",  
    baseUrl:  
    "https://api.groq.com/openai/v1",  
    model: "openai/gpt-oss-20b",  
    keyUrl:  
    "https://console.groq.com/keys",  
  },  
  gemini: {  
    label: "Google Gemini (free  
tier)",  
    type: "gemini",  
    baseUrl:  
    "https://generativelanguage.googleapis.com/v1beta",  
    model: "gemini-flash-latest",  
    keyUrl:  
    "https://aistudio.google.com/apikey",
```

```
    },
    custom: {
      label: "Other (OpenAI-
compatible)",
      type: "openai",
      baseUrl: "",
      model: "",
      keyUrl: "",
    },
  };

const STORAGE_KEY =
"jarvis.settings.v1";

export function defaultSettings() {
  return { provider: "groq", lang:
"", providers: {} };
}

function getStorage() {
  try { return
globalThis.localStorage || null; }
catch { return null; }
}

export function loadSettings(storage
= getStorage()) {
  const base = defaultSettings();
  try {
```

```

    const raw = storage &&
storage.getItem(STORAGE_KEY);
    if (!raw) return base;
    const data = JSON.parse(raw);
    if (!data || typeof data !==
"object") return base;
    return {
        provider:
PROVIDER_PRESETS[data.provider] ?
data.provider : base.provider,
        lang: typeof data.lang ===
"string" ? data.lang : "",
        providers: data.providers &&
typeof data.providers === "object" ?
data.providers : {},
    };
} catch {
    return base; // corrupt storage -
> start fresh instead of crashing
}
}

export function
saveSettings(settings, storage =
getStorage()) {
    try {
        if (!storage) return false;
        storage.setItem(STORAGE_KEY,
JSON.stringify(settings));

```

```

        return true;
    } catch {
        return false;
    }
}

// Each provider keeps its own key,
// so a Groq key is never sent to Google
// (or vice versa).
export function
getProviderSettings(settings, name =
settings.provider) {
    const preset =
PROVIDER_PRESETS[name];
    const saved = (settings.providers
&& settings.providers[name]) || {};
    return {
        type: preset.type,
        apiKey: saved.apiKey || "",
        model: saved.model ||
preset.model,
        baseUrl: saved.baseUrl ||
preset.baseUrl,
    };
}

export function speechLang(settings)
{
    return settings.lang ||

```

```
(globalThis.navigator &&  
navigator.language) || "en-US";  
}
```